

**INSTITUTO
FEDERAL**

Santa Catarina

Câmpus
São José

Coordenação de tarefas

Sistemas Operacionais

Bernardo Souza Muniz.

08 de Maio de 2026

Engenharia de Telecomunicações - IFSC-SJ

Sumário

1. Introdução	3
2. Alternância de Uso	4
3. Algoritmo de Peterson	6
4. TSL	8
5. Conclusão	9

1. Introdução

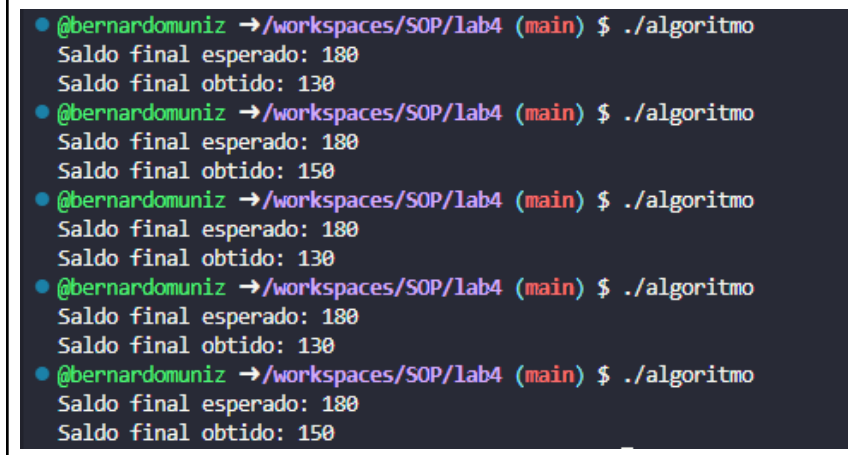
Este trabalho tem como objetivo apresentar a resolução dos problemas relacionados à condição de corrida (*Race Condition*) entre tarefas, abordados na disciplina de Sistemas Operacionais.

Foi disponibilizado o seguinte código para a implementação dos algoritmos de Alternância de Uso, Algoritmo de Peterson e TSL (*Test-and-Set Lock*).

```
1  int saldo = 100;
2  void* depositar(void* arg) {
3      int valor = *(int*)arg;
4      int temp = saldo;
5      usleep(100);
6      temp += valor;
7      saldo = temp;
8      return NULL;
9  }
10 int main() {
11     pthread_t t1, t2;
12     int v1 = 50, v2 = 30;
13     pthread_create(&t1, NULL, depositar, &v1);
14     pthread_create(&t2, NULL, depositar, &v2);
15     pthread_join(t1, NULL);
16     pthread_join(t2, NULL);
17     printf("Saldo final esperado: 180\n");
18     printf("Saldo final obtido: %d\n", saldo);
19     return 0;
20 }
```

Ao executar o código, é possível verificar uma incoerência nos resultados obtidos, visto que o saldo inicial da conta é de 100 reais e são realizados dois depósitos, de 50 e 30 reais, respectivamente. Assim, o valor esperado ao final da execução seria de 180 reais. Entretanto, observa-se que o saldo final varia entre 130 e 150 reais, evidenciando um problema de condição de corrida.

Figura 1: Elaborada pelo Autor



```
• @bernardomuniz →/workspaces/SOP/lab4 (main) $ ./algoritmo
Saldo final esperado: 180
Saldo final obtido: 130
• @bernardomuniz →/workspaces/SOP/lab4 (main) $ ./algoritmo
Saldo final esperado: 180
Saldo final obtido: 150
• @bernardomuniz →/workspaces/SOP/lab4 (main) $ ./algoritmo
Saldo final esperado: 180
Saldo final obtido: 130
• @bernardomuniz →/workspaces/SOP/lab4 (main) $ ./algoritmo
Saldo final esperado: 180
Saldo final obtido: 130
• @bernardomuniz →/workspaces/SOP/lab4 (main) $ ./algoritmo
Saldo final esperado: 180
Saldo final obtido: 150
```

Execução do algoritmo apresentado

Isso ocorre devido à condição de corrida entre as tarefas, na qual mais de uma thread manipula simultaneamente um recurso compartilhado, tornando o resultado final imprevisível. O objetivo da implementação dos algoritmos é eliminar a condição de corrida, garantindo o resultado correto da operação.

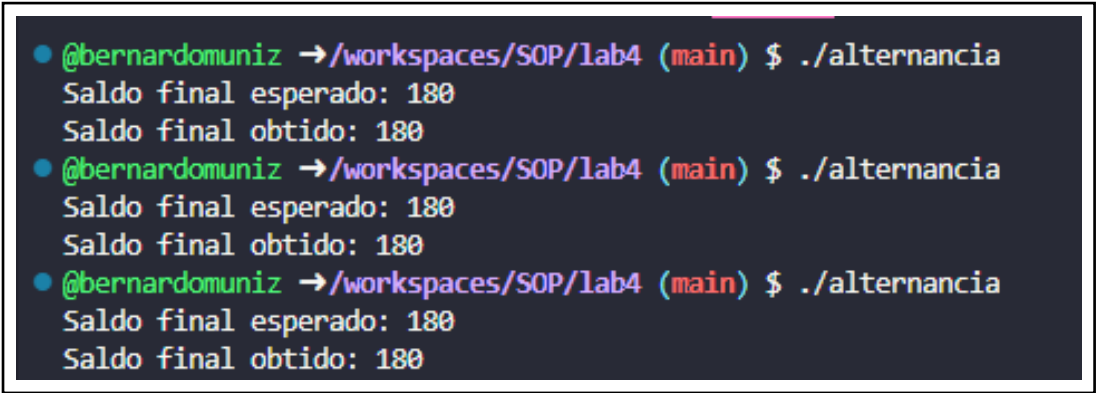
2. Alternância de Uso

Ao realizar a implementação do algoritmo de alternância de uso, foi obtido o seguinte resultado:

```
1  #include <stdio.h>
2  #include <pthread.h>
3  #include <unistd.h>
4
5  int saldo = 100;
6  int num_tasks = 2;
7  int turn = 0;
8
9  void enter(int task) {
10     while (turn != task){
11     }
12 }
13
14 void leave(){
15     turn = (turn + 1) % num_tasks;
16 }
17
18 void* depositar(void* arg) {
19     int task_id = *(int*)arg;
20     int valor = (task_id == 0) ? 50 : 30;
21
22     enter(task_id);
23     int temp = saldo;
24     usleep(100);
25     temp += valor;
26     saldo = temp;
27
28     leave(task_id);
29     return NULL;
30 }
31
32 int main() {
33     pthread_t t1, t2;
34     int index_t1 = 0, index_t2 = 1;
35
36     pthread_create(&t1, NULL, depositar, &index_t1);
37     pthread_create(&t2, NULL, depositar, &index_t2);
38
39     pthread_join(t1, NULL);
40     pthread_join(t2, NULL);
41
42     printf("Saldo final esperado: 180\n");
43     printf("Saldo final obtido: %d\n", saldo);
44
45     return 0;
46 }
```

Ao executar o programa mais de uma vez foi possível verificar que a condição de corrida foi sanada:

Figura 2: Elaborada pelo Autor



```
• @bernardomuniz →/workspaces/SOP/lab4 (main) $ ./alternancia
  Saldo final esperado: 180
  Saldo final obtido: 180
• @bernardomuniz →/workspaces/SOP/lab4 (main) $ ./alternancia
  Saldo final esperado: 180
  Saldo final obtido: 180
• @bernardomuniz →/workspaces/SOP/lab4 (main) $ ./alternancia
  Saldo final esperado: 180
  Saldo final obtido: 180
```

Execução do algoritmo de Alternância de uso

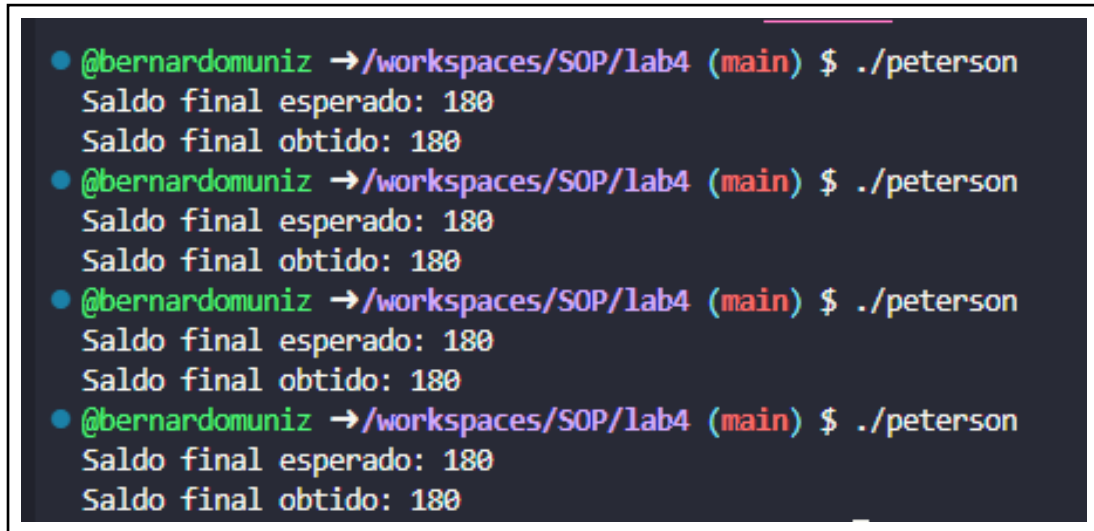
3. Algoritmo de Peterson

Ao realizar a implementação do algoritmo de Peterson, foi obtido o seguinte resultado:

```
1  #include <stdio.h>
2  #include <pthread.h>
3  #include <unistd.h>
4
5  int turn = 0;
6  int wants[2] = {0, 0};
7  int saldo = 100;
8
9  void enter(int task) {
10     int other = 1 - task;
11     wants[task] = 1;
12     turn = other;
13     while ((turn == other) && wants[other]) {
14     }
15 }
16
17 void leave(int task) {
18     wants[task] = 0;
19 }
20
21 void* depositar(void* arg) {
22     int task_id = *(int*)arg;
23     int valor = (task_id == 0) ? 50 : 30;
24     enter(task_id);
25
26     int temp = saldo;
27     usleep(100);
28     temp += valor;
29     saldo = temp;
30     leave(task_id);
31
32     return NULL;
33 }
34 int main() {
35     pthread_t t1, t2;
36     int v1 = 50, v2 = 30;
37     int id_task1 = 0, id_task2 = 1;
38
39     pthread_create(&t1, NULL, depositar, &id_task1);
40     pthread_create(&t2, NULL, depositar, &id_task2);
41
42     pthread_join(t1, NULL);
43     pthread_join(t2, NULL);
44
45     printf("Saldo final esperado: 180\n");
46     printf("Saldo final obtido: %d\n", saldo);
47     return 0;
48 }
```

Ao executar o programa mais de uma vez foi possível verificar que a condição de corrida foi sanada:

Figura 3: Elaborada pelo Autor

A terminal window with a dark background and light-colored text. It shows four separate executions of a program named 'peterson'. Each execution starts with a prompt from the user '@bernardomuniz' followed by the command '→/workspaces/SOP/lab4 (main) \$./peterson'. The output of each execution is two lines: 'Saldo final esperado: 180' and 'Saldo final obtido: 180'. The text is color-coded: the prompt is green, the path and command are purple, and the output is white.

```
• @bernardomuniz →/workspaces/SOP/lab4 (main) $ ./peterson
  Saldo final esperado: 180
  Saldo final obtido: 180
• @bernardomuniz →/workspaces/SOP/lab4 (main) $ ./peterson
  Saldo final esperado: 180
  Saldo final obtido: 180
• @bernardomuniz →/workspaces/SOP/lab4 (main) $ ./peterson
  Saldo final esperado: 180
  Saldo final obtido: 180
• @bernardomuniz →/workspaces/SOP/lab4 (main) $ ./peterson
  Saldo final esperado: 180
  Saldo final obtido: 180
```

Execução do algoritmo de Peteron

4. TSL

Ao realizar a implementação do algoritmo de TSL, foi obtido o seguinte resultado:

```
1  #include <stdio.h>
2  #include <pthread.h>
3  #include <unistd.h>
4
5
6  int saldo = 100;
7  int lock = 0; // variável de trava
8
9  void enter(int *lock){ // passa o endereço da trava
10     while ( __sync_lock_test_and_set(lock,1) ) {
11         // espera ocupada sobre a trava
12     }
13 }
14
15 void leave(int *lock) {
16     *lock = 0; // libera a seção crítica
17 }
18
19 void* depositar(void* arg) {
20     int task_id = *(int*)arg;
21     int valor = (task_id == 0) ? 50 : 30;
22     enter(&lock);
23
24     int temp = saldo;
25     usleep(100);
26     temp += valor; // Passo 2: Soma
27     saldo = temp; // Passo 3: Escrita
28     leave(&lock);
29     return NULL;
30 }
31
32 int main() {
33     pthread_t t1, t2;
34     int index_t1 = 0, index_t2 = 1;
35
36     pthread_create(&t1, NULL, depositar, &index_t1);
37     pthread_create(&t2, NULL, depositar, &index_t2);
38
39     pthread_join(t1, NULL);
40     pthread_join(t2, NULL);
41
42     printf("Saldo final esperado: 180\n");
43     printf("Saldo final obtido: %d\n", saldo);
44     return 0;
45 }
```


Ao executar o programa mais de uma vez foi possível verificar que a condição de corrida foi sanada:

Figura 4: Elaborada pelo Autor

A terminal window with a dark background and light-colored text. It shows four identical lines of command execution. Each line starts with a blue bullet point, followed by the user '@bernardomuniz', the directory '/workspaces/SOP/lab4', the shell '(main)', and the command '\$./tsl'. Below each command, two lines of output are shown: 'Saldo final esperado: 180' and 'Saldo final obtido: 180'.

```
• @bernardomuniz →/workspaces/SOP/lab4 (main) $ ./tsl
  Saldo final esperado: 180
  Saldo final obtido: 180
• @bernardomuniz →/workspaces/SOP/lab4 (main) $ ./tsl
  Saldo final esperado: 180
  Saldo final obtido: 180
• @bernardomuniz →/workspaces/SOP/lab4 (main) $ ./tsl
  Saldo final esperado: 180
  Saldo final obtido: 180
• @bernardomuniz →/workspaces/SOP/lab4 (main) $ ./tsl
  Saldo final esperado: 180
  Saldo final obtido: 180
```

Execução do algoritmo de TSL

5. Conclusão

A implementação dos algoritmos de sincronização, como Alternância de Uso, Peterson e TSL (Test-and-Set Lock), verificou-se a importância do controle de acesso à região crítica, garantindo que apenas uma thread manipule o recurso compartilhado por vez. Dessa forma, foi possível garantir o valor correto dos dados e alcançar o objetivo proposto.